

Session 16: Pandas Series - Full Lecture Notes

Data Science Mentorship Program (DSMP) 2022-23 | CampusX

1. Introduction to Pandas

Pandas is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation tool built on top of the Python programming language.

- **Why Pandas?** In machine learning, before building models, you must understand the data. Pandas allows you to perform "data autopsy"—analyzing, cleaning, and manipulating data to find patterns.
- **Key Objects:**
 1. **Series:** A 1D array-like object (similar to a column in a table).
 2. **DataFrame:** A 2D table-like structure (multiple series joined together).

2. The Pandas Series

A Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.).

Setup

```
import numpy as np
import pandas as pd
```

2.1 Creating a Series from a List

```
# String Series
country = ['India', 'Pakistan', 'USA', 'Nepal', 'Srilanka']
pd.Series(country)

# Integer Series
runs = [13, 24, 56, 78, 100]
pd.Series(runs)

# Custom Index
marks = [67, 57, 89, 100]
subjects = ['Maths', 'English', 'Science', 'Hindi']
marks_series = pd.Series(marks, index=subjects, name="Nitish's Marks")
print(marks_series)
```

Output:

```
Maths    67
English  57
Science  89
Hindi    100
Name: Nitish's Marks, dtype: int64
```

2.2 Creating a Series from a Dictionary

When a dictionary is passed, the keys automatically become the index.

```
marks_dict = {
    'Maths': 67,
    'English': 57,
    'Science': 89,
    'Hindi': 100
}
pd.Series(marks_dict)
```

3. Series Attributes

Attributes give information about the Series object without changing it.

Attribute	Description
.size	Returns the number of items.
.dtype	Returns the data type of values.
.name	Returns the name of the series.
.is_unique	Returns True if all values are unique.
.index	Returns the index object.
.values	Returns the data as a Numpy array.

4. Reading External Data (read_csv)

In real-world scenarios, we read data from CSV files. By default, `read_csv` returns a `DataFrame`. Use `squeeze=True` to force it into a `Series` if it's a single column.

1. Single Column Data (Subscribers)

```
subs = pd.read_csv('subs.csv', squeeze=True)
```

2. Multi-column Data (Kohli IPL Runs)

We set Match Number as Index and use Runs as values

```
vk = pd.read_csv('kohli_ipl.csv', index_col='match_no', squeeze=True)
```

3. Bollywood Movies (Movie Name as Index, Actor as Value)

```
movies = pd.read_csv('bollywood.csv', index_col='movie', squeeze=True)
```

5. Important Series Methods

5.1 Previewing Data

- `head(n)`: Returns first n rows (default 5).
- `tail(n)`: Returns last n rows (default 5).
- `sample(n)`: Returns n random rows (useful for checking bias).

5.2 Frequency Counts

- `value_counts()`: Counts the frequency of unique values.

Which actor did the most movies?

```
movies.value_counts()
```

Result: Akshay Kumar (48), Amitabh Bachchan (45)...

5.3 Sorting

- `sort_values()`: Sorts by the values (e.g., sort Kohli's runs from high to low).
- `sort_index()`: Sorts by the index (e.g., sort movies alphabetically).
- **In-place parameter**: Set `inplace=True` to permanently change the original series.

6. Mathematical & Statistical Methods

- `count()`: Total non-missing items.
- `sum()`: Sum of all items.
- `mean()`, `median()`, `mode()`: Central tendency.
- `std()`, `var()`: Dispersion.
- `min()`, `max()`: Extremes.

- `describe()`: Returns a summary (count, mean, std, min, 25%, 50%, 75%, max).

7. Indexing & Slicing

Indexing

- **Positive Indexing:** `vk[0]` fetches the first item.
- **Negative Indexing:** Works only if the index consists of strings (Labels). If the index is numeric, negative indexing may throw an error.
- **Label Indexing:** `movies['2 States']` returns 'Alia Bhatt'.

Slicing

- `vk[5:16]`: Returns matches 6 to 16.
- `vk[-5:]`: Returns last 5 matches.
- **Fancy Indexing:** `vk[[1, 3, 5]]` returns runs in match 1, 3, and 5.

8. Boolean Indexing (Filtering)

This is the most powerful feature. You create a boolean mask and apply it to the Series.

```
# Find matches where Kohli scored a 50+
vk[vk >= 50]
```

```
# Count number of 50s
vk[vk >= 50].size
```

```
# Find movies of Akshay Kumar
movies[movies == 'Akshay Kumar']
```

9. Series Arithmetic & Operations

- **Broadcasting:** You can perform operations on the whole series at once.
 - `100 - marks_series`: Subtracts every mark from 100.
- **Comparison:** `vk >= 50` returns a Series of True/False values.

10. Plotting

Pandas allows for quick visualization.

```
# Line plot of subscriber growth
subs.plot()
```

```
# Bar chart of top 20 actors with most movies
movies.value_counts().head(20).plot(kind='bar')
```

```
# Pie chart of movie distribution
movies.value_counts().head(10).plot(kind='pie')
```

11. Summary

The lecture concludes that while everything done today could technically be done in Python or Numpy, Pandas provides a high-level abstraction specifically for tabular data analysis.

Understanding **Series** is the foundation for mastering **DataFrames**, which are essentially collections of Series objects.